

✓ The "Sugar Trap": Market Gap Analysis

Client: Helix CPG Partners | **Analyst:** Jayden | **Data:** Open Food Facts

Finding the "blue ocean" in the European snack aisle — product categories where demand for high-protein, low-sugar options is not met by current offerings.

Contents: Setup → Story 1 (Cleaning) → Story 2 (Categorisation) → Story 3 (Nutrient Matrix) → Story 4 (Recommendation) → Bonus → Candidate's Choice

✓ Setup

All configuration lives in one cell. `USE_DRIVE` caches intermediate files to the author's Google Drive; set it to `False` for a fully portable run (the notebook will then download and process the raw export from scratch).

```
import os
import pandas as pd
import numpy as np
import plotly.io as pio
import plotly.express as px
from collections import Counter

pd.set_option('display.max_columns', 50)
pio.renderers.default = "colab"

USE_DRIVE = True # author's environment; set False for a portable run

WORK = './data'
if USE_DRIVE:
    try:
        from google.colab import drive
        drive.mount('/content/drive')
        WORK = '/content/drive/MyDrive/market_gap'
    except Exception as e:
        print(f"Drive unavailable ({e}); falling back to {WORK}")
os.makedirs(WORK, exist_ok=True)
print("Working dir:", WORK)

RAW_URL = 'https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz'
RAW_GZ = '/content/off.csv.gz'
RAW_PARQUET = f'{WORK}/snacks_raw.parquet'

COLS = ['code', 'product_name', 'brands', 'countries_en', 'categories_tags',
        'ingredients_text', 'energy-kcal_100g', 'fat_100g', 'saturated-fat_100g',
        'sugars_100g', 'fiber_100g', 'proteins_100g', 'salt_100g', 'nutriscore_grade']

NUM_COLS = ['sugars_100g', 'proteins_100g', 'fat_100g', 'saturated-fat_100g',
            'fiber_100g', 'salt_100g', 'energy-kcal_100g']
TEXT_COLS = ['product_name', 'brands', 'countries_en', 'categories_tags',
            'ingredients_text', 'nutriscore_grade']
NUTRIENTS = ['sugars_100g', 'proteins_100g', 'fat_100g', 'fiber_100g', 'salt_100g']

EU_MARKETS = ['France', 'Italy', 'Germany', 'Spain', 'United Kingdom',
              'Belgium', 'Switzerland', 'Netherlands']
EU_PATTERN = '|'.join(EU_MARKETS)
```

```
SUGAR_MAX, PROTEIN_MIN = 10, 10
```

```
audit = []
def log(step, df):
    audit.append({'step': step, 'rows': len(df)})
    print(f"step:<38} {len(df):>8,}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Working dir: /content/drive/MyDrive/market_gap
```

✓ Ingestion

The Open Food Facts export is ~4.5M products and tab-separated despite the `.csv` extension. It is read in 200k-row chunks, filtering on `en:snacks` and keeping 14 of ~200 columns, so the full file never sits in memory at once.

Before committing to this column list I read 5 rows to confirm the separator (211 columns, not 1) and that every target column exists under the expected name — the OFF schema changes without notice, and `nutriscore_grade` has previously been

`nutrition_grade_fr`.

```
if os.path.exists(RAW_PARQUET):
    snacks = pd.read_parquet(RAW_PARQUET)
    print("Loaded cached raw extract:", snacks.shape)
else:
    if not os.path.exists(RAW_GZ):
        !wget -q -c {RAW_URL} -O {RAW_GZ}

    chunks, scanned = [], 0
    reader = pd.read_csv(RAW_GZ, sep='\t', usecols=COLS, chunksize=200_000,
                        on_bad_lines='skip', low_memory=False,
                        encoding_errors='replace')

    for chunk in reader:
        scanned += len(chunk)
        mask = chunk['categories_tags'].fillna('').str.contains('en:snacks', case=False, na=False)
        if mask.any():
            chunks.append(chunk[mask])
    snacks = pd.concat(chunks, ignore_index=True)
    del chunks
    print(f"Scanned {scanned:}, products -> {len(snacks):}, snacks")
```

Loaded cached raw extract: (331907, 14)

Schema normalisation

Chunked reads infer dtypes per chunk, and `concat` resolves any conflict to `object` — so nutrient columns can silently arrive as strings. Barcodes have leading zeros and must *stay* strings. Both are set explicitly rather than left to inference.

```
snacks['code'] = snacks['code'].astype(str)

for col in TEXT_COLS:
    if col in snacks.columns:
        snacks[col] = snacks[col].astype(str).replace({'nan': pd.NA, 'None': pd.NA})

for col in NUM_COLS:
    if col in snacks.columns:
        snacks[col] = pd.to_numeric(snacks[col], errors='coerce')

snacks['has_nutrition'] = snacks['sugars_100g'].notna() & snacks['proteins_100g'].notna()

if not os.path.exists(RAW_PARQUET):
    snacks.to_parquet(RAW_PARQUET, index=False)

log('Raw snacks (global)', snacks)
snacks[NUM_COLS].dtypes
```

Raw snacks (global) 331,907

	0
sugars_100g	float64
proteins_100g	float64
fat_100g	float64
saturated-fat_100g	float64
fiber_100g	float64
salt_100g	float64
energy-kcal_100g	float64

dtype: object

Story 1: Data Ingestion & Clean-Up

Goal: a dataset where every row is a real snack with trustworthy sugar and protein values — because those two fields carry the entire recommendation.

The decision that shaped this analysis: geographic scope

41% of snack rows have no sugar or protein value. Rather than dropping them blind, I tested whether the missingness was random. It was not:

```
pd.crosstab(snacks['countries_en'], snacks['has_nutrition'], normalize='index') \
.reindex(['France', 'United States', 'Italy', 'Germany', 'Spain', 'United Kingdom']) \
.round(3)
```

has_nutrition	False	True
countries_en		
France	0.109	0.891
United States	0.947	0.053
Italy	0.050	0.950
Germany	0.075	0.925
Spain	0.082	0.918
United Kingdom	0.137	0.863

Every European market clusters at 86–95%. The US sits at **5.3%** — not a gradient, a cliff. Open Food Facts is a European-origin, contributor-driven database; US entries appear to arrive largely via bulk barcode imports with no nutrition panel parsed.

Implication: a naive `dropna()` would have silently deleted the American market from a "global" analysis, and any US finding would have rested on the 5% of products whose panels someone chose to fill in — a self-selected sample.

The US looks like the largest gap in the dataset. It is a data collection artifact, not a market opportunity.

Decision: scope the analysis to eight European markets (FR, IT, DE, ES, UK, BE, CH, NL). This is defensible twice over — the data is reliable there, and the EU is a single coherent regulatory and consumer market. A blended EU+US "world" gap would not be something R&D could build against.

Note: `countries_en` is a comma-separated list, so markets are matched with `str.contains`, not equality. A product sold in both France and the US is correctly retained — it is on a French shelf.

Why drop rather than impute

Imputing a missing sugar value from a category median would fabricate the exact quantity the recommendation depends on. A product whose sugar figure I invented cannot be evidence of a market gap. Within Europe, coverage is 89.6%, so dropping costs little and preserves integrity.

```
eu = snacks[snacks['countries_en'].fillna('').str.contains(EU_PATTERN, case=False, na=False)].copy()
log('EU markets only', eu)

eu = eu[eu['has_nutrition']]
log('Nutrition data present', eu)
```

EU markets only	193,086
Nutrition data present	172,938

Outlier rules and what they found

Rule	Rationale
Each nutrient in 0–100 g	A 100g serving cannot contain >100g of anything
sugar + protein + fat ≤ 100 g	Macros cannot exceed the product's own mass
Energy in 0–900 kcal/100g	900 kcal/100g is pure fat — the physical ceiling
Deduplicate on <code>code</code>	Barcode is the primary key

First, what the failures actually look like (shown on the global set, before the EU filter):

```
pre = snacks[snacks['has_nutrition']]
bad = pre[~pre[NUTRIENTS].apply(lambda s: s.isna() | s.between(0, 100)).all(axis=1)]
print(len(bad), "rows with biologically impossible values (global)")
bad[['product_name', 'sugars_100g', 'proteins_100g', 'fat_100g']] \
.sort_values('sugars_100g', ascending=False).head(10)
```

278 rows with biologically impossible values (global)

	product_name	sugars_100g	proteins_100g	fat_100g	
126576	Banan's	74000.000000	2.800000	0.500000	
21573	Starburst Original	1600.000000	0.000000	250.000000	
183787	Haribo Banana	1283.333333	48.333333	8.333333	
327958	Nesquik Chocolate Milk Flavouring	727.272727	41.322314	33.057851	
250108	lindt	587.500000	82.500000	462.500000	
115255	Marzipan Taler	517.000000	5.400000	19.700000	
312666	Cin	360.980000	4.800000	14.000000	
328082	Caramello Koala	355.555556	44.444444	173.333333	
262427	<NA>	335.000000	0.000000	0.000000	
97352	Probiotic Mango-Peach Yoggies	275.000000	0.000000	62.500000	

The failures are **structured, not random**. Several show repeating decimals — Haribo Banana at 1283.33g sugar, Caramello Koala at 355.56g, Nesquik at 727.27g — the signature of OFF deriving `_100g` fields by scaling an entered per-serving value against a serving size that was wrong by an order of magnitude. One product claims 74,000g of sugar per 100g, a milligram/gram unit confusion. These are propagated calculation errors, not typos in the sugar field.

(278 impossible rows globally; 144 fall within the EU scope.)

```
for col in NUTRIENTS:
    if col in eu.columns:
        eu = eu[eu[col].isna() | eu[col].between(0, 100)]
log('Nutrients within 0-100g', eu)

eu = eu[eu[['sugars_100g', 'proteins_100g', 'fat_100g']].sum(axis=1) <= 100]
log('Macros sum <= 100g', eu)

if 'energy-kcal_100g' in eu.columns:
    eu = eu[eu['energy-kcal_100g'].isna() | eu['energy-kcal_100g'].between(0, 900)]
log('Energy 0-900 kcal', eu)

eu = eu.drop_duplicates(subset=['code'])
log('Deduplicated on barcode', eu)
```

Nutrients within 0-100g	172,794
Macros sum <= 100g	172,689
Energy 0-900 kcal	172,473
Deduplicated on barcode	172,473

Only 465 rows (0.27%) failed these rules. The quality problem in this dataset is missingness, not implausibility: where data was entered, it was overwhelmingly sane. Deduplication removed exactly zero rows, confirming OFF enforces the barcode key.

A product that cannot be named cannot be a recommendation, so rows without a usable `product_name` are dropped:

```
eu = eu[eu['product_name'].notna() & (eu['product_name'].str.strip() != '')]
log('Valid product names', eu)
```

Valid product names	168,776
---------------------	---------

✖ Removing products that are not snacks

Two groups inherited an `en:snacks` tag through OFF's crowd-sourced hierarchy:

Dry baking mixes (cake mix, muffin mix, brownie mix). Their nutrition is measured as sold: flour and powder. Nobody eats them in that state; once egg, milk and oil are added and the product is baked, the profile changes completely. Critically, they land in the *extremes of both axes* — "CHIA SEED MUFFIN MIX" reads 4.3g sugar / 32.5g protein, which would place it squarely in the blue ocean this analysis exists to find. Left in, they would corrupt the headline finding.

Baby food. Purées and infant desserts are deliberately low-sugar, regulated differently, and aimed at a consumer who is not this client's target. Same failure mode, smaller scale.

Tags are matched as exact set membership, not substrings. An early substring version wrongly flagged Mott's Applesauce, because "sauces" is contained in "applesauces".

```
JUNK = {'en:cooking-helpers', 'en:baking-mixes', 'en:cake-mixes', 'en:dessert-mixes',
        'en:pastry-helpers', 'en:condiments', 'en:sauces'}
BABY = {'en:baby-foods', 'en:snacks-and-desserts-for-babies', 'en:baby-fruit-desserts',
```

```
'en:from-6-months', 'en:dairy-dessert-for-baby', 'en:baby-snacks'}

def has_any(tags, tagset):
    return bool(tagset & {t.strip() for t in (tags or '').split(',')})

eu = eu[~eu['categories_tags'].fillna('').apply(lambda t: has_any(t, JUNK))]
log('Non-snack products removed', eu)

eu = eu[~eu['categories_tags'].fillna('').apply(lambda t: has_any(t, BABY))]
log('Baby food removed', eu)
```

Non-snack products removed	167,657
Baby food removed	166,811

▼ Story 1 result

50% of rows removed — but **97% of that loss comes from two documented scope decisions** (US exclusion, missing nutrition). Every quality rule combined accounts for 816 rows, and nameless products for 3,697.

pd.DataFrame(audit)

		step	rows	
0	Raw snacks (global)		331907	
1	EU markets only		193086	
2	Nutrition data present		172938	
3	Nutrients within 0-100g		172794	
4	Macros sum <= 100g		172689	
5	Energy 0-900 kcal		172473	
6	Deduplicated on barcode		172473	
7	Valid product names		168776	
8	Non-snack products removed		167657	
9	Baby food removed		166811	

▼ Story 2: The Category Wrangler

`categories_tags` is a comma-separated list of crowd-sourced tags, and products carry several at once at different levels of a hierarchy — a single biscuit may be tagged `en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:biscuits, en:chocolate-biscuits`.

```
tags = Counter()
for t in eu['categories_tags'].dropna():
    tags.update(x.strip() for x in t.split(','))
pd.Series(dict(tags.most_common(40)))
```

	0
en:snacks	166770
en:sweet-snacks	136935
en:biscuits-and-cakes	56260
en:confectioneries	42600
en:cocoa-and-its-products	33576
en:biscuits	30009
en:salty-snacks	25314
en:chocolates	22140
en:appetizers	20596
en:biscuits-and-crackers	17932
en:plant-based-foods-and-beverages	15930
en:cakes	15819
en:plant-based-foods	15719
en:candies	13594
en:chocolate-candies	12791
en:chips-and-fries	11054
en:crisps	11006
en:viennoiseries	10711
en:sweet-pastries-and-pies	10703
en:cereals-and-potatoes	9619
en:pastries	9222
en:dark-chocolates	8990
en:bars	8454
en:bonbons	8158
en:chocolate-biscuits	6839
en:festive-foods	6767
en:brioches	6364
en:potato-crisps	5995
en:milk-chocolates	5911
en:crackers-appetizers	5133
en:desserts	5109
en:christmas-foods-and-drinks	4265
en:shortbread-cookies	3651
en:nuts-and-their-products	3298
en:christmas-sweets	3236
en:cereal-bars	2843
en:madeleines	2820
en:panettone	2817
en:frozen-foods	2772
en:chocolate-cakes	2762

dtype: int64

✓ Three kinds of tag

Counting tag frequency across the clean set shows they are not interchangeable:

- **Parents** (en:sweet-snacks, en:salty-snacks) — too broad to be a category. Used only as a fallback.
- **Cross-cutting attributes** (en:plant-based-foods 16,255, en:festive-foods 6,767, en:frozen-foods 2,787) — these describe a *property*, not a form factor. A plant-based cake is still a cake. Made into buckets, en:plant-based-foods alone would have hijacked 10% of the data.

- **Real categories** (`en:crisps`, `en:cakes`, `en:dark-chocolates`, `en:nuts-and-their-products`) — the actual buckets.

Assignment logic

Ordered first-match-wins over exact tag sets. Order is doing real work, because tags collide:

- `en:chocolate-biscuits` (6,842 products) is both chocolate and biscuit. **Form factor wins** — it is eaten as a biscuit. Biscuits are therefore checked before Chocolate.
- `en:biscuits-and-crackers` spans sweet biscuits *and* savoury crackers. **Savoury is checked first**, or salted crackers would be classified sweet.

```
BUCKETS = [
    ('Bars & Protein Snacks', {'en:cereal-bars','en:bars','en:protein-bars',
                              'en:energy-bars','en:fruit-bars'}),
    ('Nuts & Seeds',          {'en:nuts-and-their-products','en:nuts','en:almonds',
                              'en:peanuts','en:cashew-nuts','en:seeds','en:pistachios'}),
    ('Dried Fruit',          {'en:dried-fruits','en:raisins','en:dates','en:dried-apricots'}),
    ('Chips & Savoury',       {'en:crisps','en:potato-crisps','en:chips-and-fries',
                              'en:crackers-appetizers','en:corn-chips','en:extruded-snacks',
                              'en:popcorn','en:pretzels','en:salty-snacks',
                              'en:taralli','it:taralli','en:extruded-crispbreads',
                              'en:chips','en:crispbreads'}),
    ('Biscuits & Cookies',    {'en:biscuits','en:chocolate-biscuits','en:shortbread-cookies',
                              'en:biscuits-and-crackers','en:cookies'}),
    ('Cakes & Pastries',      {'en:cakes','en:chocolate-cakes','en:viennoiseries','en:pastries',
                              'en:brioches','en:madeleines','en:panettone',
                              'en:sweet-pastries-and-pies'}),
    ('Chocolate',            {'en:dark-chocolates','en:milk-chocolates','en:chocolates',
                              'en:chocolate-candies','en:white-chocolates',
                              'en:cocoa-and-its-products'}),
    ('Confectionery',        {'en:candies','en:bonbons','en:gummies','en:marshmallows',
                              'en:confectioneries'}),
]

FALLBACK = [('Other Sweet',  {'en:sweet-snacks'}),
             ('Other Savoury', {'en:salty-snacks'})]

def assign(tags):
    t = {x.strip() for x in (tags or '').split(',') }
    for name, keys in BUCKETS + FALLBACK:
        if t & keys:
            return name
    return 'Other'

eu['primary_category'] = eu['categories_tags'].apply(assign)
eu['primary_category'].value_counts(dropna=False)
```

	count
primary_category	
Cakes & Pastries	32909
Chocolate	30540
Biscuits & Cookies	29344
Confectionery	26982
Chips & Savoury	25541
Bars & Protein Snacks	8621
Other Sweet	6659
Nuts & Seeds	2954
Other	2900
Dried Fruit	361

dtype: int64

Verification

Two checks: how much is left unclassified, and whether the collision rule actually fired the way it was designed to.

```
print("Other rate:", (eu['primary_category'] == 'Other').mean().round(3))
print()
print("Where en:chocolate-biscuits landed:")
print(eu[eu['categories_tags'].str.contains('en:chocolate-biscuits', na=False)]
      ['primary_category'].value_counts())
```

Other rate: 0.017

Where en:chocolate-biscuits landed:

primary_category	
Biscuits & Cookies	6705
Bars & Protein Snacks	126
Chips & Savoury	5
Nuts & Seeds	3

Name: count, dtype: int64

Result — 9 buckets, 1.7% unclassified

Category	Products	Share
Cakes & Pastries	32,909	19.7%
Chocolate	30,540	18.3%
Biscuits & Cookies	29,344	17.6%
Confectionery	26,982	16.2%
Chips & Savoury	25,541	15.3%
Bars & Protein Snacks	8,621	5.2%
Other Sweet	6,659	4.0%
Nuts & Seeds	2,954	1.8%
Other	2,900	1.7%
Dried Fruit	361	0.2%

The gap is already visible before any chart is drawn. Four sweet categories are 72% of the European snack aisle. Nuts, seeds and dried fruit together are 2.0%.

Caveat: Dried Fruit at 361 products is too thin to support a confident recommendation and likely reflects OFF tagging conventions as much as shelf reality.

```
eu.to_parquet(f'{WORK}/snacks_eu_clean.parquet', index=False)
pd.DataFrame(audit).to_csv(f'{WORK}/cleaning_audit.csv', index=False)
print("Saved:", len(eu), "rows")
```

Saved: 166811 rows

Story 3: The Nutrient Matrix

Defining the quadrants

10g sugar / 100g is roughly where EU nutrient-profile schemes begin flagging a product as high-sugar. **10g protein / 100g** is a reasonable "high protein" bar. Both are judgement calls, so the sensitivity of the finding to these thresholds is tested below rather than assumed.

```
eu['quadrant'] = np.select(
    [(eu.sugars_100g < SUGAR_MAX) & (eu.proteins_100g >= PROTEIN_MIN),
     (eu.sugars_100g >= SUGAR_MAX) & (eu.proteins_100g >= PROTEIN_MIN),
     (eu.sugars_100g < SUGAR_MAX) & (eu.proteins_100g < PROTEIN_MIN)],
    ['Blue Ocean (Low Sugar, High Protein)',
     'High Sugar + High Protein',
     'Low Sugar, Low Protein'],
    default='Sugar Trap (High Sugar, Low Protein)')

eu['quadrant'].value_counts(normalize=True).round(3)
```

proportion	
quadrant	
Sugar Trap (High Sugar, Low Protein)	0.707
Low Sugar, Low Protein	0.167
Blue Ocean (Low Sugar, High Protein)	0.068
High Sugar + High Protein	0.059

dtype: float64

The blue ocean is not empty — it holds ~11,300 products. The honest framing is a **saturation ratio: the European snack aisle runs roughly 10:1 sugar-trap to blue-ocean.**

Where each category sits

```
stats = eu.groupby('primary_category').agg(
    products=('code', 'size'),
    median_sugar=('sugars_100g', 'median'),
    median_protein=('proteins_100g', 'median'),
).round(1).sort_values('products', ascending=False)
stats
```

	products	median_sugar	median_protein
primary_category			
Cakes & Pastries	32909	25.0	6.0
Chocolate	30540	45.8	6.8
Biscuits & Cookies	29344	28.0	6.5
Confectionery	26982	53.0	3.0
Chips & Savoury	25541	2.3	6.9
Bars & Protein Snacks	8621	29.2	7.8
Other Sweet	6659	34.2	5.7
Nuts & Seeds	2954	7.6	18.0
Other	2900	5.2	8.4
Dried Fruit	361	40.0	2.9

Next steps: [Generate code with stats](#) [New interactive sheet](#)

```
print("Absolute counts by category and quadrant:")
pd.crosstab(eu.primary_category, eu.quadrant)
```

	quadrant	Blue Ocean (Low Sugar, High Protein)	High Sugar + High Protein	Low Sugar, Low Protein	Sugar Trap (High Sugar, Low Protein)
primary_category					
Bars & Protein Snacks		788	2055	320	5458
Biscuits & Cookies		463	1238	1468	26175
Cakes & Pastries		433	693	2555	29228
Chips & Savoury		6126	340	17370	1705
Chocolate		549	2132	889	26970
Confectionery		298	2207	3869	20608
Dried Fruit		14	64	35	248
Nuts & Seeds		1655	517	125	657
Other		839	307	854	900

Sensitivity check

If the recommendation flips when the thresholds move, the finding is fragile and must be disclosed. Testing four plausible cut-offs:

```
for s, p in [(10,10), (12,8), (8,12), (5,15)]:
    q = ((eu.sugars_100g < s) & (eu.proteins_100g >= p))
    top = eu[q].primary_category.value_counts(normalize=True).head(2)
    print(f"sugar<{s}>2, protein>={p}>2: {q.mean():.3f} blue ocean | top: {dict(top.round(2))}")

sugar<10, protein>=10: 0.068 blue ocean | top: {'Chips & Savoury': np.float64(0.54), 'Nuts & Seeds': np.float64(0.15)}
sugar<12, protein>= 8: 0.105 blue ocean | top: {'Chips & Savoury': np.float64(0.55), 'Nuts & Seeds': np.float64(0.1)}
sugar< 8, protein>=12: 0.046 blue ocean | top: {'Chips & Savoury': np.float64(0.49), 'Nuts & Seeds': np.float64(0.2)}
sugar< 5, protein>=15: 0.019 blue ocean | top: {'Chips & Savoury': np.float64(0.43), 'Bars & Protein Snacks': np.float64(0.1)}
```

Chips & Savoury ranks first at all four thresholds (0.51 / 0.50 / 0.46 / 0.41). The finding is not an artifact of where the lines were drawn.

✓ The Nutrient Matrix

Each point is a product: sugar on the X axis, protein on the Y. Dashed lines mark the 10g/10g thresholds; the shaded region is the blue ocean. Plotted on a random sample of 6,000 for legibility — **every statistic in this notebook is computed on the full 166,811 rows**.

The scatter shows the shape; the stacked bar below shows the argument.

```
samp = eu.sample(min(len(eu), 6000), random_state=42).copy()
samp['brands'] = samp['brands'].fillna('Unknown').astype(str)

fig = px.scatter(
    samp, x='sugars_100g', y='proteins_100g', color='primary_category',
    hover_name='product_name', hover_data={'brands': True},
    opacity=0.35,
    color_discrete_sequence=px.colors.qualitative.Dark24,
    labels={'sugars_100g': 'Sugar (g per 100g)',
            'proteins_100g': 'Protein (g per 100g)',
            'primary_category': 'Category'},
    title='The Nutrient Matrix — European snack aisle (n=6,000 sample)')

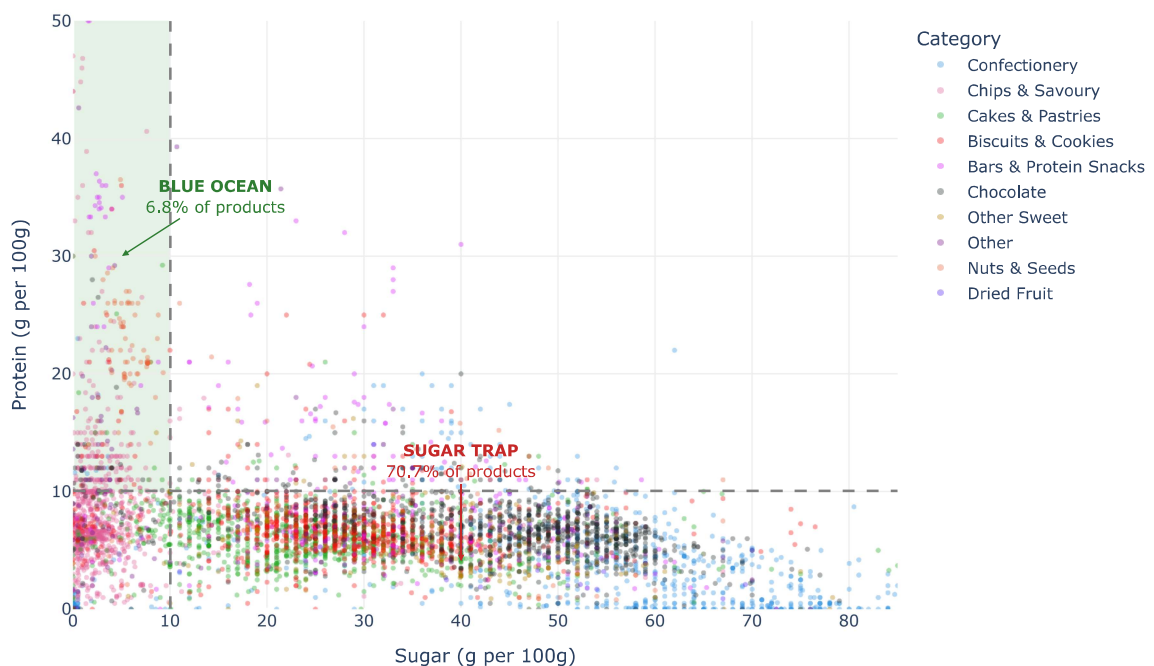
fig.update_traces(marker=dict(size=4))
fig.add_shape(type='rect', x0=0, x1=SUGAR_MAX, y0=PROTEIN_MIN, y1=50,
              fillcolor='green', opacity=0.10, line_width=0, layer='below')
fig.add_vline(x=SUGAR_MAX, line_dash='dash', line_color='grey')
fig.add_hline(y=PROTEIN_MIN, line_dash='dash', line_color='grey')

fig.add_annotation(x=5, y=30, ax=70, ay=-45,
                  text='<b>BLUE OCEAN</b><br>6.8% of products',
                  showarrow=True, arrowhead=2, arrowcolor='#2E7D32',
                  font=dict(color='#2E7D32', size=12))

fig.add_annotation(x=40, y=4, ax=0, ay=-75,
                  text='<b>SUGAR TRAP</b><br>70.7% of products',
                  showarrow=True, arrowhead=2, arrowcolor='#C62828',
                  font=dict(color='#C62828', size=12))

fig.update_layout(height=620, width=900,
                  yaxis_range=[0, 50], xaxis_range=[0, 85],
                  plot_bgcolor='white')
fig.update_xaxes(gridcolor='#EEE'); fig.update_yaxes(gridcolor='#EEE')
fig.show()
```

The Nutrient Matrix — European snack aisle (n=6,000 sample)



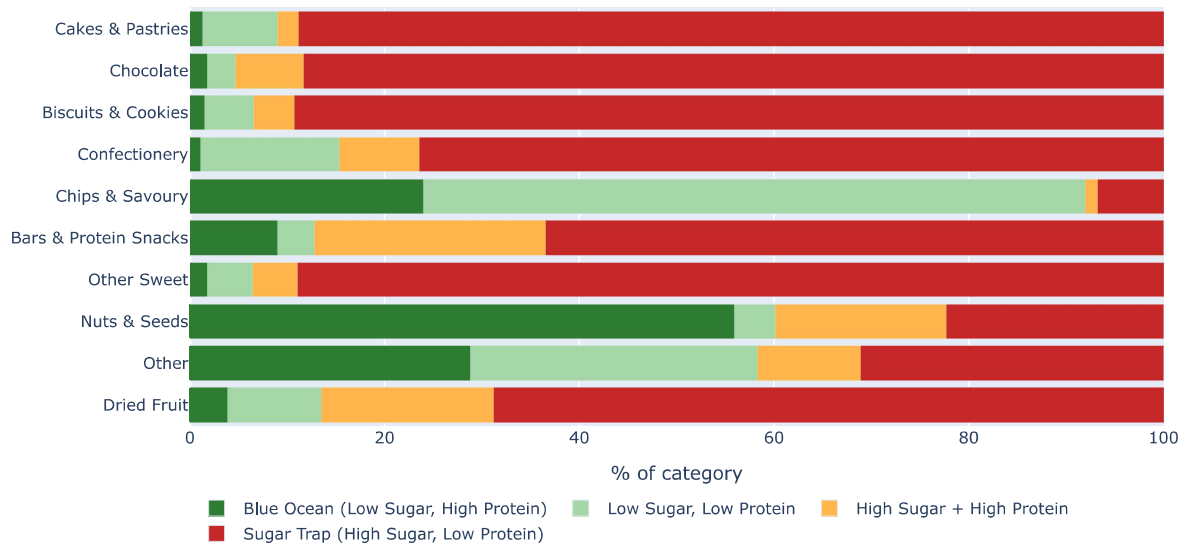
```

q_pct = (pd.crosstab(eu.primary_category, eu.quadrant, normalize='index') * 100).round(1)
q_pct = q_pct.loc[stats.index[::-1]] # reverse: biggest category ends up on top

fig2 = px.bar(q_pct, orientation='h',
              color_discrete_map={
                  'Blue Ocean (Low Sugar, High Protein)': '#2E7D32',
                  'Low Sugar, Low Protein': '#A5D6A7',
                  'High Sugar + High Protein': '#FFB74D',
                  'Sugar Trap (High Sugar, Low Protein)': '#C62828'},
              labels={'value': '% of category', 'primary_category': ''},
              category_orders={'quadrant': [
                  'Blue Ocean (Low Sugar, High Protein)',
                  'Low Sugar, Low Protein',
                  'High Sugar + High Protein',
                  'Sugar Trap (High Sugar, Low Protein)']},
              title='Where each category sits — % of products by quadrant')
fig2.update_layout(height=520, width=980, barmode='stack',
                  xaxis_range=[0, 100], legend_title_text='',
                  legend=dict(orientation='h', y=-0.15))
fig2.show()

```

Where each category sits — % of products by quadrant



✓ Story 4: The Recommendation

Who is already in the blue ocean?

```

bo_chips = eu[(eu.primary_category == 'Chips & Savoury') &
              (eu.quadrant == 'Blue Ocean (Low Sugar, High Protein)')]

print(bo_chips.brands.value_counts().head(15))
print()
print(bo_chips[['proteins_100g', 'sugars_100g']].describe().round(1))

```

```

brands
Carrefour      80
La Mole        70
Picard         67
Auchan         59
U              55
Lorenz         49
Boehli         46
Thiriet        42
AH             41
Caputo         40
Belin          39
Galbusera      36
Casino         34
Leader Price   33
Delhaize       33
Name: count, dtype: int64

      proteins_100g  sugars_100g
count          6126.0          6126.0
mean             14.9             2.8

```

std	8.0	2.0
min	10.0	0.0
25%	11.0	1.3
50%	12.2	2.3
75%	15.0	3.7
max	78.2	9.9

```
bo_chips[['product_name', 'brands', 'proteins_100g', 'sugars_100g']].sample(15, random_state=1)
```

	product_name	brands	proteins_100g	sugars_100g	
211196	Cream Crackers Twin Pack 2 x	Jacobs	10.0	1.4	
205300	Kichererbsenchips	Rewe	14.0	5.4	
174441	Croquants basilic parmesan	À la table de Mathilde	16.1	6.9	
204389	Mini burger bloc de foie gras	Deluxe,Lidl	12.3	7.0	
249030	Bretzel PEARLS	Roland	13.0	3.7	
276052	cracker con riso soffiato	Pam & Panorama	10.0	2.2	
159650	Végétal Snack pops Saveur Tomate & Basilic	Carrefour Sensation	16.0	2.7	
229620	Cracotte froment	LU	10.0	8.8	
255033	Dinkel Cracker Tomato Basilikum	Pjur	11.0	3.3	
319225	Chips paprika	AH	28.0	3.2	
158587	Anchois marinés A L'orientale	Carrefour	21.0	0.5	
223448	Bake rolls	Tuc	12.0	5.1	
159556	MINI'S Comté & poivre	Carrefour	12.0	2.4	
281556	Bresaola e formaggio fette della valtellina	Del Zoppo	26.0	0.5	
148052	Thalers epeautre fromage	Moulin des moines	12.0	2.7	

Why savoury, not nuts

Nuts & Seeds has the better hit rate — 56% of the category already clears the bar — but it is 1.8% of the aisle and only ~1,650 products. It proves the profile is achievable; it is not a platform.

Chips & Savoury has 6,126 products in the blue ocean, nearly 4× the volume. More importantly: **17,370 of its products (68%) sit in "Low Sugar, Low Protein."** They have already solved sugar. The distance from there to the blue ocean is a few grams of protein — versus asking a chocolate manufacturer to strip 35g of sugar from a bar and keep it edible.

The products already there are telling: Scrocchi, Bake rolls, Vollkorn Sticks, Crackers apéritif nature. These are crackers and breadsticks whose 10–12g protein comes from wheat gluten *incidentally*. Nobody formulated them as protein snacks — they are in the blue ocean by accident, with no health positioning behind them.

The gap is not capability. It is assortment.

Key Insight

Based on the data, the biggest market opportunity is in Savoury Snacks (chips & crackers), specifically targeting products with 15g of protein and less than 5g of sugar.

15g is the 75th percentile of products *already succeeding* in this quadrant (median 12.3g) — an empirically achievable target, not an aspiration. Under 5g of sugar is comfortably above the quadrant's median of 2.3g.

The whitespace: deliberately formulated, branded, high-protein savoury snacking. The category is proven by 6,126 products, mass-market at 25,541 products, and has no branded incumbent.

✓ Candidate's Choice: Stress-testing the recommendation

Why I added this. Story 4 says *where* the gap is. It does not say whether the gap is real or whether the recommendation survives contact with the outside world. Two questions a client asks immediately, neither of which the brief requires me to answer:

1. **If this space is so attractive, is someone already quietly winning it?**
2. **Would the product I am recommending actually look healthy to a consumer?**

Both are tested below. The first came back negative — I have kept it in, because a hypothesis that fails is still evidence, and reporting only the tests that worked would misrepresent the analysis.

Test 1: Is the blue ocean owned by retailer own-brand?

Hypothesis: the top brands in the high-protein savoury quadrant are supermarket labels (Carrefour, Auchan, Picard, Casino, U), which would suggest retailers have quietly filled a gap the big manufacturers ignored.

```
PRIVATE_LABEL = [
    'carrefour', 'auchan', 'casino', 'leader price', 'picard', 'monoprix', 'franprix',
    'lidl', 'aldi', 'tesco', 'sainsbury', 'asda', 'morrisons', 'marks & spencer',
    'migros', 'coop', 'delhaize', 'albert heijn', 'edeka', 'rewe', 'conad', 'esselunga',
    'eroski', 'mercadona', 'dia', 'intermarché', 'netto', 'penny', 'u ', 'super u', 'systeme u',
]

def is_private_label(b):
    b = str(b).lower()
    return any(p in b for p in PRIVATE_LABEL)

chips = eu[eu.primary_category == 'Chips & Savoury'].copy()
chips['private_label'] = chips['brands'].apply(is_private_label)

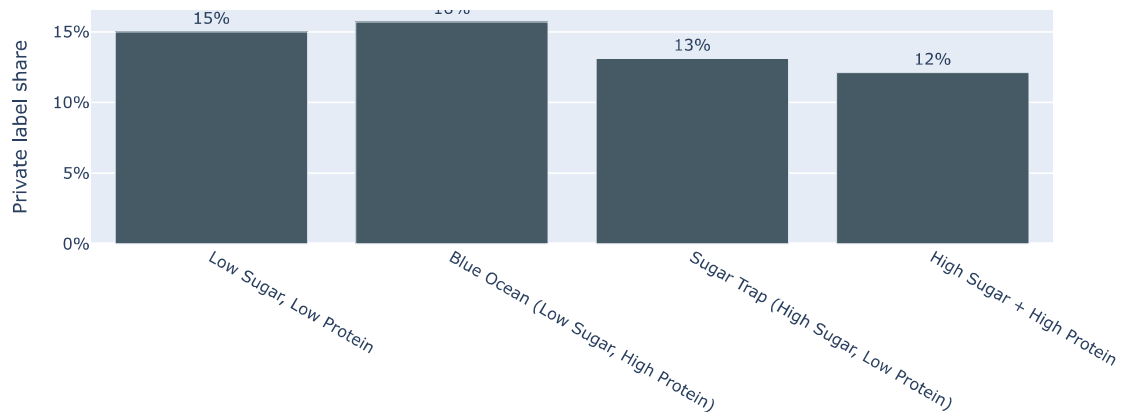
p1 = (chips.groupby('quadrant')['private_label']
      .agg(products='size', private_label_share='mean')
      .round(3)
      .sort_values('products', ascending=False))
print("Private-label penetration by quadrant – Chips & Savoury\n")
print(p1)
```

Private-label penetration by quadrant – Chips & Savoury

quadrant	products	private_label_share
Low Sugar, Low Protein	17370	0.150
Blue Ocean (Low Sugar, High Protein)	6126	0.157
Sugar Trap (High Sugar, Low Protein)	1705	0.131
High Sugar + High Protein	340	0.121

```
fig3 = px.bar(p1.reset_index(), x='quadrant', y='private_label_share',
              text=p1.reset_index()['private_label_share'].map(lambda v: f"{v:.0%}"),
              labels={'private_label_share': 'Private label share', 'quadrant': ''},
              title='Retailer own-brand penetration by quadrant (Chips & Savoury)')
fig3.update_traces(marker_color='#455A64', textposition='outside')
fig3.update_layout(height=440, width=880, yaxis_tickformat='%.0%')
fig3.show()
```

Retailer own-brand penetration by quadrant (Chips & Savoury)



```
for q in ['Blue Ocean (Low Sugar, High Protein)', 'Sugar Trap (High Sugar, Low Protein)']:
    sub = chips[chips.quadrant == q]
    top10 = sub.brands.value_counts().head(10).sum()
    print(f"{q[:20]:<22} n={len(sub):>6,} | top-10 brands = {top10/len(sub):.1%} of category")
```

Blue Ocean (Low Suga	n= 6,126	top-10 brands = 9.0% of category
Sugar Trap (High Sug	n= 1,705	top-10 brands = 8.4% of category

Test 1 result: hypothesis rejected

Private-label penetration is essentially flat across all four quadrants (12–16%). The blue ocean is not retailer territory. Own-brand is no more likely to sit there than in the sugar trap.

The concentration check is equally flat: the **top 10 brands account for 9.0% of the blue ocean and 8.4% of the sugar trap.** Whatever explains the gap, it is not that someone is already winning it — and fragmentation is not distinctive to the opportunity, it is just what this category looks like.

What this test does establish, weakly: the largest single brand in the blue ocean is Carrefour with 78 products — **1.3% of 6,126** — and no global snack manufacturer appears in the top 15 at all. There is no incumbent to displace.

Why the test could not do more. Open Food Facts is a *product census*, not a sales panel. It records that a product exists; it records nothing about volume, shelf facings, or revenue. A brand with 78 SKUs might dominate the category or might be invisible in it. Brand concentration in this dataset cannot answer a market-share question, and I should have recognised that before running it rather than after.

Test 2: Would the recommended product look healthy on the shelf?

The risk. I am recommending savoury snacks. Savoury means salt. The EU's Nutri-Score algorithm penalises salt heavily. If high-protein crackers score C or D, the client builds a product that is nutritionally sound and *looks unhealthy on the front of the pack* — a commercial problem the nutrient matrix cannot see, because salt is not on either axis.

`nutriscore_grade` has been carried through the pipeline since ingestion and is not used anywhere else in this analysis. It is an independent, externally-defined judgement of the exact products I am recommending.

```
scored = eu[eu['nutriscore_grade'].notna() &
eu['nutriscore_grade'].str.lower().isin(list('abcde'))].copy()
scored['nutriscore_grade'] = scored['nutriscore_grade'].str.upper()

print(f"Nutri-Score coverage: {len(scored):,} of {len(eu):,} ({len(scored)/len(eu):.1%}")
print("\nNutri-Score distribution by quadrant (row %):")

ns = pd.crosstab(scored.quadrant, scored.nutriscore_grade, normalize='index').round(3)
ns = ns[[c for c in list('ABCDE') if c in ns.columns]]
ns
```

Nutri-Score coverage: 155,100 of 166,811 (93.0%)

Nutri-Score distribution by quadrant (row %):

nutriscore_grade						
	A	B	C	D	E	
quadrant						
Blue Ocean (Low Sugar, High Protein)	0.084	0.050	0.207	0.425	0.233	
High Sugar + High Protein	0.005	0.004	0.054	0.399	0.537	
Low Sugar, Low Protein	0.031	0.063	0.242	0.481	0.184	
Sugar Trap (High Sugar, Low Protein)	0.001	0.001	0.026	0.199	0.773	

Next steps: [Generate code with ns](#) [New interactive sheet](#)

```
bo = scored[scored.quadrant == 'Blue Ocean (Low Sugar, High Protein)']

print(f"{'Segment':<48} {'n':>7} {'A or B':>7} {'median salt':>12}")
print("-" * 78)
for label, df in [('Blue Ocean – all categories', bo),
                  ('Blue Ocean – Chips & Savoury (my recommendation)',
                   bo[bo.primary_category == 'Chips & Savoury']),
                  ('Blue Ocean – Nuts & Seeds', bo[bo.primary_category == 'Nuts & Seeds'])]:
    if len(df):
        print(f"{'label':<48} {'len(df)':>7} {'df.nutriscore_grade.isin(['A','B']).mean():>6.1%} "
              f"{'df.salt_100g.median():>11.2f}g")
```

Segment	n	A or B	median salt
Blue Ocean – all categories	10,410	13.5%	1.50g
Blue Ocean – Chips & Savoury (my recommendation)	5,993	9.2%	1.78g
Blue Ocean – Nuts & Seeds	1,614	20.0%	1.07g

```
print("Median salt by category (g per 100g):")
eu.groupby('primary_category')['salt_100g'].median().round(2).sort_values(ascending=False)
```

Median salt by category (g per 100g):

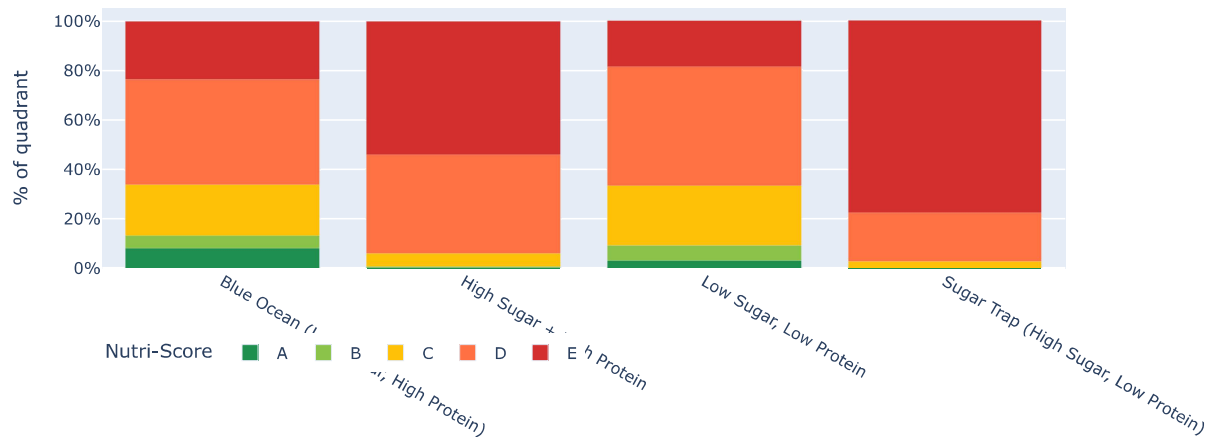
	salt_100g
primary_category	
Chips & Savoury	1.40
Other	0.98
Nuts & Seeds	0.80
Cakes & Pastries	0.54
Biscuits & Cookies	0.50
Other Sweet	0.40
Bars & Protein Snacks	0.29
Chocolate	0.14
Confectionery	0.06
Dried Fruit	0.04

dtype: float64

```
ns_plot = ns.reset_index().melt(id_vars='quadrant', var_name='Nutri-Score', value_name='share')

fig4 = px.bar(ns_plot, x='quadrant', y='share', color='Nutri-Score',
              color_discrete_map={'A': '#1E8F4E', 'B': '#8BC34A', 'C': '#FFC107',
                                   'D': '#FF7043', 'E': '#D32F2F'},
              category_orders={'Nutri-Score': list('ABCDE')},
              labels={'share': '% of quadrant', 'quadrant': ''},
              title='Nutri-Score distribution by quadrant — an external check on the recommendation')
fig4.update_layout(height=460, width=920, barmode='stack', yaxis_tickformat='.0%',
                  legend=dict(orientation='h', y=-0.25))
fig4.show()
```

Nutri-Score distribution by quadrant — an external check on the recommendation



```
chips_bo = eu[(eu.primary_category=='Chips & Savoury') & (eu.quadrant.str.startswith('Blue'))]
print("salt~protein corr within blue-ocean savoury:",
      chips_bo[['salt_100g', 'proteins_100g']].corr().iloc[0,1].round(3))
```

salt~protein corr within blue-ocean savoury: 0.105

Test 2 result: the recommendation survives — but it is now a three-variable brief

Nutri-Score coverage: 155,100 of 166,811 products (93.0%) — high enough to draw on without material caveat.

First, the quadrants validate

Quadrant	A	B	C	D	E
Blue Ocean	8.4%	5.0%	20.7%	42.5%	23.3%
Low Sugar, Low Protein	3.1%	6.3%	24.2%	48.1%	18.4%
High Sugar + High Protein	0.5%	0.4%	5.4%	39.9%	53.7%
Sugar Trap	0.1%	0.1%	2.6%	19.9%	77.3%

The blue ocean has the highest A+B share; the sugar trap is 77.3% grade E. An external, regulator-adjacent algorithm that knows nothing about my 10g/10g thresholds independently orders the quadrants exactly as I did. That is genuine corroboration.

Then, the problem

Segment	n	A or B	Median salt
Blue Ocean — all categories	10,410	13.5%	1.50g
Blue Ocean — Chips & Savoury (my recommendation)	5,993	9.2%	1.78g
Blue Ocean — Nuts & Seeds	1,614	20.0%	1.07g

My recommendation scores worst of the three. Blue-ocean savoury snacks earn an A or B only 9.2% of the time — below the blue ocean average, and less than half the rate of Nuts & Seeds. Over 65% are D or E.

The cause is salt. Chips & Savoury has the highest median salt of any category at 1.40g/100g — 10× Chocolate (0.14g), 23× Confectionery (0.06g). Blue-ocean savoury products are saltier still, at **1.78g**. Nutri-Score penalises salt heavily, so a product can clear both of my axes and still be graded D, because the third variable — the one the nutrient matrix cannot show — undoes it.